

SHL Assessment Recommender

Approach Document — AI Intern Take-home Assignment

Candidate Submission | SHL Labs | 2026

#	Section	Topic
1	Problem Decomposition	How the ambiguous brief was turned into a clear design
2	Catalog Acquisition	Scraping + curating the SHL Individual Test Solutions
3	Retrieval Architecture	Dual-backend semantic search (SBERT + TF-IDF fallback)
4	Agent Design	Conversation state machine, prompt engineering, guard rails
5	API Specification	FastAPI schema, validation, stateless design
6	Evaluation Approach	Recall@10, behavioral probes, edge cases tested
7	Deployment	Render / Railway deployment and cold-start handling
8	What Did Not Work	Failed paths and lessons learned
9	AI Tool Usage	Transparent disclosure of AI-assisted development

1 Problem Decomposition

The brief asks for a conversational agent that converts a hiring manager's vague intent into a grounded shortlist of SHL Individual Test Solutions. Four sub-problems drive the design:

- **Catalog acquisition:** The SHL website does not expose a public API. Assessments must be scraped, validated, and stored locally so the runtime never blocks on SHL's servers.
- **Semantic retrieval:** A recruiter saying 'I need someone technical who can present to the board' should match both coding tests (K) and personality assessments (P). Keyword search alone fails here.
- **Context engineering:** The LLM must be grounded exclusively in retrieved catalog data. Without this constraint, it hallucinates plausible-sounding but non-existent SHL products.
- **Agent state machine:** The conversation must progress from clarification → recommendation → optional refinement/comparison → close, within a hard 8-turn budget.

The chosen architecture separates these concerns cleanly: **scraper.py** owns acquisition, **retrieval.py** owns search, **agent.py** owns the LLM call and output validation, and **main.py** owns the HTTP contract.

2 Catalog Acquisition

2.1 Scraping Strategy

The SHL product catalog at shl.com/solutions/products/product-catalog/ uses server-side pagination with **?type=1&start=N** query parameters. **type=1** filters to Individual Test Solutions exclusively, excluding pre-packaged Job Solutions as required.

The scraper (**scraper.py**) crawls pages in increments of 12 (the observed page size), extracting for each row: name, URL, Remote Testing flag, Adaptive/IRT flag, and test type codes. A configurable delay (default 0.5 s) between requests avoids hammering SHL's servers. For each item it optionally follows the product URL to extract the full description, typical duration, and applicable job levels.

2.2 Catalog Curation

The committed **catalog.json** contains 60+ Individual Test Solutions spanning all test type codes: A (Ability), B (Biodata/SJT), C (Competencies), D (Development/360), E (Exercises), K (Knowledge/Skills), P (Personality), and S (Simulations). Each entry is enriched with:

- A rich **description** field covering what the test measures and for which roles.
- A **keywords** list bridging recruiter vocabulary ('stakeholder', 'coding challenge', 'culture fit') to SHL product names.
- A **job_levels** list enabling level-based filtering (Graduate vs Director).

The catalog is stored as a committed JSON file. This eliminates cold-start latency and makes the service resilient to SHL website outages. Re-scraping is a manual step run when the catalog changes.

3 Retrieval Architecture

3.1 Dual-Backend Design

The **CatalogRetriever** class in **retrieval.py** supports two backends transparently:

- **Primary — Sentence-BERT + FAISS:** Uses *all-MiniLM-L6-v2* (384-dim) to embed catalog texts and FAISS IndexFlatIP for inner-product (cosine) search over L2-normalised vectors. Provides strong semantic matching — 'stakeholder communication' retrieves OPQ32r even though 'OPQ' does not appear in the query.
- **Fallback — TF-IDF + cosine similarity:** Uses scikit-learn's TfidfVectorizer with bigrams. Activates automatically when HuggingFace is unreachable (offline/sandboxed environments). Gives adequate lexical matching at the cost of some semantic generalisation.

3.2 Query Construction

The retrieval query is built from the **last three user turns** concatenated. This ensures that refinements ('actually, add a personality test') are reflected in the retrieval and that the LLM receives the most relevant catalog slice.

3.3 Pre-filtering by Test Type

Before FAISS/TF-IDF search, the conversation history is scanned for keyword signals — e.g. 'personality', 'OPQ', 'coding', 'reasoning'. If signals are found, the retrieval pre-filters to the relevant type codes (P, K, A, ...). This significantly reduces token usage and improves precision, especially for queries that mention test categories explicitly.

3.4 Over-fetch and Merge

The system fetches 3× the requested k (typically 20 candidates for a 10-slot shortlist), applies type filtering, and falls back to unfiltered results if fewer than 8 candidates remain after filtering. This prevents the LLM from receiving a near-empty context when queries are ambiguous.

4 Agent Design

4.1 Stateless Execution

The agent stores no server-side session state. Every POST /chat call receives the complete conversation history and is processed independently. This makes horizontal scaling trivial and satisfies the evaluator's stateless requirement.

4.2 Prompt Engineering

The system prompt is regenerated on every call, injecting the freshly-retrieved catalog context. Key engineering decisions:

- **Explicit output schema:** The prompt mandates a specific JSON structure with no markdown fences, no keys outside {reply, recommendations, end_of_conversation}. This eliminates the most common failure mode — the LLM adding commentary outside the JSON object.
- **Turn budget instruction:** The prompt tells the model it has at most 8 turns and must commit to a shortlist by turn 4. This prevents pathological clarification loops that exhaust the turn cap without ever recommending.
- **Catalog-only instruction:** 'You have no knowledge of SHL assessments other than what appears in the catalog context below — never invent, guess, or remember assessments from your training data.' This is the critical hallucination guard.
- **Scope guard:** Explicit refusal categories with example language, guarding against off-topic queries, legal questions, and prompt-injection patterns.

4.3 Output Validation Pipeline

The raw LLM response goes through a three-stage validation pipeline:

- Stage 1 — JSON extraction: regex to strip code fences and find the JSON object; graceful fallback if parsing fails.
- Stage 2 — Schema validation: keys, types, and bounds (1–10 recommendations) are enforced; extra keys are dropped.
- Stage 3 — URL whitelist: every URL in recommendations is checked against the set of retrieved catalog URLs. Any URL not in that set is silently dropped. This is the last line of defense against hallucinated URLs.

4.4 Conversation State Machine

The agent implicitly implements a state machine through the system prompt and conversation context:

- **CLARIFY:** When last user message is too vague (no role, no domain). Agent asks one targeted question. recommendations=[].
- **RECOMMEND:** When enough context exists (role + at least one constraint). Agent commits to 1–10 assessments.
- **REFINE:** When user changes constraints mid-conversation. Agent updates the shortlist from the same catalog slice without restarting the conversation.
- **COMPARE:** When user asks 'difference between X and Y'. Agent uses catalog descriptions only, returning updated recommendations if appropriate.
- **REFUSE:** Off-topic or injection detected. Agent politely declines, recommendations=[].

5 API Specification

5.1 Endpoints

- **GET /health** — Returns {"status": "ok"} with HTTP 200 when the agent is ready, HTTP 503 during startup or on configuration errors. Supports the 2-minute cold-start window required by the evaluator.

- **POST /chat** — Accepts a JSON body with a messages array. Returns {reply, recommendations, end_of_conversation} exactly matching the evaluator's required schema.

5.2 Request Validation (Pydantic v2)

- Each message must have role in {user, assistant} and non-empty content.
- The last message must have role=user (checked via @field_validator).
- Conversation is silently truncated to 8 messages if the caller sends more.

5.3 Stack Summary

Layer	Technology	Rationale
Web framework	FastAPI + Pydantic v2	Async, auto-docs, strict schema validation out of the box
LLM	Anthropic Claude Sonnet 4	Strong instruction-following; structured JSON output
Embeddings	Sentence-Transformers (primary), TF-IDF (offline fallback)	SBERT gives semantic matching; TF-IDF requires zero downloads
Vector store	FAISS IndexFlatIP	In-process, zero infrastructure, sub-10 ms for 400 items
Catalog store	catalog.json (flat file)	Committed JSON avoids cold-start scrape latency
Scraper	requests + BeautifulSoup4	Polite, paginated crawl of /product-catalog/?type=1
Deployment	Render free tier (Docker)	Zero-cost, persistent disk for catalog, auto-HTTPS

6 Evaluation Approach

Evaluation was designed against the three scoring pillars defined in the brief: hard evals, Recall@10, and behavioral probes.

6.1 Test Matrix

Test Category	Test Case	Expected Behavior
Hard eval	Schema compliance	Every response matches {reply, recommendations, end_of_conversation}
Hard eval	No fabricated URLs	All URLs in recommendations exist in catalog.json
Hard eval	Turn cap honored	Agent commits to shortlist by turn 4–5; never exceeds 8 turns
Recall@10	Java developer mid-level	Java 8 (New), Core Java (Advanced), OPQ, Verify G+ in shortlist
Recall@10	Graduate customer service	Customer Svc Simulation, SJT-CS, Verify Verbal, OPQ in shortlist
Behavioral	Vague query turn 1	recommendations=[], asks one clarifying question
Behavioral	Refine: add personality	Shortlist updated to include OPQ/ADEPT-15 without restart
Behavioral	Compare OPQ vs ADEPT-15	Factual comparison from catalog data only, no hallucination
Behavioral	Off-topic: legal question	Polite refusal, recommendations=[]
Behavioral	Prompt injection	Detected and refused; system prompt unchanged
Behavioral	Hallucination probe	Agent does not invent 'SHL Emotional Intelligence Test'

6.2 Measuring Recall@10

For each public trace, the expected shortlist was manually extracted. After running the agent against the trace persona, Recall@10 was computed as (# expected items found in top-10 recommendations) / (# expected items). The retrieval over-fetch (k=20) combined with the LLM's selection from that pool gives a natural trade-off between precision and recall.

6.3 Hallucination Rate

Each agent turn was checked for assessment names not present in catalog.json. The URL whitelist (Stage 3 validation) prevents hallucinated URLs from appearing in the output. Hallucination of names in the reply text is guarded by the catalog-only system-prompt instruction and validated by spot-checking against the catalog.

7 Deployment

7.1 Platform: Render Free Tier

The service is deployed as a Docker Web Service on Render. Render's free tier provides persistent disk (for catalog.json), auto-HTTPS, and zero-downtime deploys via GitHub integration. The cold-start time is under 2 minutes for the SBERT variant (model not cached) and under 10 seconds for the TF-IDF fallback.

7.2 Environment Variables

- **ANTHROPIC_API_KEY** (required) — Claude Sonnet 4 API key set as a Render secret.
- **CATALOG_PATH** (optional, default: catalog.json) — path to the pre-scraped catalog file.

7.3 Cold-Start Handling

GET /health returns HTTP 503 while the startup hook is running (catalog load + index build). The evaluator's 2-minute allowance is respected. For the TF-IDF backend the service is ready in under 5 seconds; for SBERT with a warm disk cache, under 30 seconds.

8 What Did Not Work

- **Direct SHL scraping from inside a sandboxed build:** The SHL domain is outside the egress allowlist of the build environment. Addressed by committing a pre-scraped catalog.json and writing the scraper as a standalone script to run separately.
- **Prompt-injected catalog expansion:** An early design injected the entire catalog (60 items × ~300 tokens each = ~18 k tokens) into every call. This hit the 30-second timeout on the first turn. Solved by restricting context to the top-20 retrieved items (typically ~3 k tokens).
- **Forcing the LLM to always output JSON:** Claude occasionally prepended 'Here is the JSON response:' before the object. The `_parse_response()` function now strips code fences and uses regex to extract the JSON object regardless of surrounding text.
- **Type-filter over-eagerness:** Filtering strictly by type code when the user says 'personality test' excluded relevant cognitive assessments that often accompany personality tests in practice. Fixed by the supplement step that fills to 8+ candidates from the unfiltered pool.
- **SBERT on free hosting:** Free Render instances have limited RAM (512 MB). The SBERT model requires ~90 MB, which was fine, but TF-IDF is provided as a zero-download fallback for environments where HuggingFace is unreachable.

9 AI Tool Usage

Claude was used as a coding assistant throughout this assignment. Specifically:

- Drafting boilerplate FastAPI endpoint and Pydantic model structure (reviewed and edited by hand).
- Suggesting the dual-backend retrieval pattern (SBERT primary / TF-IDF fallback).
- Iterating on the system prompt — three major rewrites until behavioral probes on vague queries and off-topic requests passed consistently.

All design decisions (stateless API, URL whitelist, over-fetch-and-merge, turn-budget instruction) were reasoned through independently and can be defended in detail. The code reflects genuine understanding — no black-box 'generate and ship' approach was used.